

RESEARCH

Open Access



CMMamba: channel mixing Mamba for time series forecasting

Qiang Li^{1,2}, Jiwei Qin^{1,2*}, Daishun Cui^{1,2†}, Dezhi Sun^{1,2†} and Dacheng Wang³

[†]Daishun Cui and Dezhi Sun contributed equally to this work.

*Correspondence:
jwqin_xju@163.com

¹ School of Computer Science and Technology, Xinjiang University, Shuimogou District, Urumqi 830046, Xinjiang Uygur Autonomous Region, China

² Xinjiang Key Laboratory of Signal Detection and Processing, Xinjiang University, Shuimogou District, Urumqi 830046, Xinjiang Uygur Autonomous Region, China

³ Institute of Remote Sensing and Digital Earth, Chinese Academy of Sciences, Haidian District, Beijing 101408, China

Abstract

Transformer-based methods have achieved excellent results in the field of time series forecasting due to their powerful ability to model sequences and capture their long-term dependencies. However, Transformer-based methods are still computationally inefficient and have high costs when modeling long sequences. In addition, previous work ignored the destruction of the original in-channel features by too many mixed features when performing channel mixing. To address the above issues, we introduce Mamba into the field of time series forecasting. The selective state-space model Mamba models long sequences more efficiently by expanding linearly according to the sequence length. Based on this, we propose Channel Mixing Mamba (CMMamba), which uses bidirectional Mamba to model sequences and uses a channel mixing mechanism to select appropriate mixed features to enhance the feature representation within the original channel. By introducing Mamba, we achieve efficient modeling of sequences. We also propose Deep Convolutional Structure (DCS) to learn cross-channel dependencies and temporal order information. Extensive experimental results on six real-world public datasets demonstrate the effectiveness of CMMamba.

Keywords: Time series, Forecasting, Channel mixing, Mamba

Introduction

Time series forecasting refers to using existing time series data to predict future values or trends, which is widely used in several real-world domains such as transportation, finance, weather, energy, etc. Earlier time series prediction methods include autoregressive (AR) [1–3], exponential smoothing [1, 4], and filter-based methods [5–7]. These methods were limited in handling high-dimensional big data and in representing complex functions efficiently [8, 9]. In recent years, with the improvement of computing power and the emergence of deep learning, deep neural networks have gradually become mainstream in time series forecasting by virtue of their ability to learn complex data representations without extensive manual feature engineering. In deep learning, the Transformers has been widely used in domains such as Natural Language Processing (NLP) [10, 11] and Computer Vision (CV) [12–14]. It is known for its powerful sequence modeling capabilities. At present, it has also become popular in time series forecasting due to these capabilities. Transformer-based methods for time series forecasting have been improving in performance. Some studies have reduced the computational

complexity [15, 16] from $O(L^2)$ to $O(L \log L)$, where L is the input sequence length. However, these methods are still computationally inefficient. They also have high computational overhead, especially when modeling long sequences. Recently, structured state-space sequence models (SSM) have emerged as a promising class of sequence modeling architectures [17, 18]. These models can be interpreted as a combination of Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN), which are inspired by classical state space models. Such models can be computed very efficiently as recursive or convolutional and have linear or near-linear scaling of sequence lengths. Albert Gu and Tri Dao [19] propose a new class of models in the selective state-space model, Mamba. Mamba improves on previous work modeling on multiple axes to achieve the modeling power of a Transformer while linearly scaling the length of the sequences. Mamba is well regarded for its strong modeling power, modeling efficiency, and low computational expense. It has been applied in CV [20–22] but has not been widely used in time series forecasting.

Real-world time series are correlated, such as an implicit correlation between the load on a transformer and the cooling oil temperature in ETT datasets [16]. In the field of time series forecasting, different series are considered different channels [23–25], and the rational use of inter-channel correlation information can improve forecasting. In previous work, Crossformer [23] segments and embeds sequences and uses attention to capture cross-channel dependencies. MCformer [25] performs channel mixing without considering the inter-channel relationships and directly stacks all the channels corresponding to the specified step interval to the target channel. These works ignored the corruption of long-term feature information of the target channel by less relevant channels. To address this problem, we propose a Mamba-based channel mixing method (CMMamba). Specifically, CMMamba first calculates the correlation of each channel with other channels and then embeds the sequences. The model learns the long-term dependent features in each channel through a bidirectional Mamba structure. Based on the correlation scores, CMMamba selects the channels with higher correlation and weightedly stacks their channel features into the target channel. This method avoids the corruption of the target channel information by channels with lower correlation while preserving the inter-channel information. In addition, we propose Deep Convolutional Structure (DCS). DCS learns long-term dependencies within sequences using depth-wise separable convolution [26]. We also use an additional set of pointwise convolution to learn cross-channel dependencies on the channel dimension. Finally, DCS preserves temporal order information on a larger time scale with the help of residual linear networks [27]. We conducted extensive experiments on six real-world public datasets to validate the effectiveness of CMMamba. The experimental results show that CMMamba not only reduces the computational expense but also maintains superior performance.

Overall, our contributions can be summarized as follows:

- We propose CMMamba, a Mamba-based channel mixing method. CMMamba extracts temporal dependencies in the channel by means of a bidirectional Mamba structure and uses correlation scores to stack highly correlated channel features to the target channel. CMMamba preserves the inter-channel information and avoids corrupting the information of the target channel by channels of lower correlation.

- We propose a deep convolutional structure. It uses a set of pointwise convolutions on top of depthwise separable convolutions to learn the channel dependencies. DCS also preserves the temporal order information with the help of residual linear networks.
- We conducted extensive experiments on six real-world publicly available datasets. These experiments show that CMMamba efficiently utilizes channel correlation information. Compared with Transformer, CMMamba is more efficient and can reduce computing costs while maintaining performance and outperforms state-of-the-art (SOTA) Transformer-based models.

The rest of the paper is structured as follows: Section "[Related work](#)" introduces, categorizes, and discusses the existing time series forecasting methods. Section "[Method](#)" introduces the Mamba structure and describes the proposed CMMamba and DCS structures in detail. Section "[Experiments](#)" describes the experimental settings, shows the experimental results, and analyzes the results in detail, demonstrating our method's effectiveness. The last section, "[Conclusion and prospect](#)" gives conclusions, potential implications, and directions for future work.

Related work

Traditional time series forecasting methods

Classical time series models are reliable tools for time series forecasting with attractive properties such as interpretability [28, 29]. Chai et al. [2] combined autoregressive integral moving average (ARIMA) and neural network with weighted fuzzy membership function as a predictive model generator, which improved the nonlinear characteristics of ARIMA and enhanced the prediction and classification performance. Shi et al. [4] proposed a penalized exponential smoothing model to predict mortality rate, which greatly reduces the number of model parameters while improving the prediction accuracy. Zhang et al. [7] propose an adaptive Kalman filtering (KF) method to eliminate the nonlinearities and stochastic uncertainties suffered from the ARIMA model of the time series, which can be adopted for air pollutant concentration prediction.

MLP-based time series forecasting models

Several studies have explored the use of Multilayer perceptron (MLP) based networks in time series forecasting [27, 30, 31]. N-BEATS [32] utilizes stacked MLP layers and double residual learning to process input data to predict the future iteratively. DEPTS [33] applied the Fourier transform to extract the period and MLP to achieve the periodic dependence of the univariate prediction. LightTS [34] uses a lightweight sample-oriented MLP structure to reduce complexity and computation time while maintaining accuracy. N-HiTS [35] combines multi-rate input sampling and layered interpolation with MLP for univariate prediction. LTSF-Linear [27] proposed a set of simple single-layer linear models to learn the time relationship between input and output sequences. At the same time, LTSF-Linear questioned the effectiveness of Transformer because LTSF-Linear achieved better performance than Transformer-based SOTA models using only simple linear layers.

Transformer-based time series forecasting models

Transformers have been successful in NLP, CV, and speech processing. Recently, many Transformer-based models have been proposed for time series forecasting tasks and have shown great potential. LogTrans [36] proposes the LogSparse attention mechanism to reduce the computational complexity of the Transformer from $O(L^2)$ to $O(L(\log L)^2)$. Informer [16] exploits the sparsity of attention scores through Kullback–Leibler divergence estimation and proposes ProbSparse self-attention, which achieves $O(L \log L)$ complexity. Autoformer [37] introduces a decomposition architecture with an autocorrelation mechanism to Transformer, achieving $O(L \log L)$ complexity. Pyraformer [38] introduces the Pyramid Attention module, which summarizes features at different resolutions and models different ranges of temporal dependencies with $O(L)$ complexity. FEDformer [39] proposes that time series have sparse representations in the frequency domain and develops a frequency-enhanced Transformer with $O(L)$ complexity. Preformer [40] divides the embedded feature vector sequence into multiple segments and utilizes segmentation correlation-based attention for prediction. Crossformer [23] introduces a cross-attention mechanism that allows the model to interact with information between time steps to help the model capture long-term dependencies in the time series. PatchTST [24] utilizes the patching of the Transformer and channel independence to efficiently improve performance, suggesting that the Transformer architecture's appropriate time series prediction Adaptation still has potential. iTransformer [41] reverses the order of information processing, first analyzing the time series information for each individual variable and then fusing the information from all the variables. MCformer [25] implements channel blending by stacking other channels at fixed intervals to the target channel and does not take into account the effect of channel correlation on the target channel.

Convolution-based time series forecasting models

Convolutional neural networks (CNNs) are widely popular for their ability to capture local features. Many CNN-based time series forecasting methods use temporal convolutional networks (TCNs) to extract local temporal dynamics [42, 43]. MICN [44] goes beyond causal convolution by proposing a multi-scale convolutional structure to combine local features and global correlations in time series. SCINet [45] eliminates the concept of causal convolution and introduces a recursive downsampling-convolution-interaction architecture to model time series with complex temporal dynamics. However, they still have difficulty in modeling long-term dependencies due to limited effective receptive fields. TimesNet [46] converts 1D time series into 2D tensors and uses a 2D kernel to capture intra-periodic and inter-periodic variations.

Other time series forecasting methods

Pham et al. [47] regarded online time series forecasting as an online, taskless continuous learning problem. And they propose a fast-and-slow learning paradigm of FSNet to handle both the fast changing and long-term knowledge in time series. FourierGNN [48] rethinks multivariate time series forecasting through a pure graph form. It uses hyper-variate graph to unify spatial and temporal dependencies, and utilizes Graph Fourier

Transform (GFT) and Discrete Fourier Transform (DFT) for efficient graph convolution operations. This approach enhances the ability to model complex spatio-temporal dependence patterns. TimeDiff [49] proposed a non-autoregressive time series forecasting method based on a conditional diffusion model, which models time series through stochastic processes and efficiently handles high-noise and complex data structures. TSDiff [50] proposed a self-guided diffusion model for probabilistic time series forecasting, which generates accurate forecasts through an iterative process of prediction, refinement, and synthesis. LS4 [51] explores deep latent state space models for time series generation, which capture the intrinsic dynamics of the time series through deep learning techniques. S-Mamba [52] introduces Mamba to time series forecasting and uses feed-forward networks to capture sequence dependencies.

Method

In this section, we describe the SSM and Mamba models in detail and then explain how CMMamba implements channel mixing. Finally, we briefly introduce the structure of the DCS.

Preliminary knowledge

Time series forecasting problem formulation. In time series forecasting, given a sequence $X = \{x_1, \dots, x_L\} \in R^{L \times M}$ of historical observations containing M channels and L time steps, the time series forecasting task can be formulated as forecasting T future time steps $Y = \{x_{L+1}, \dots, x_{L+T}\} \in R^{T \times M}$.

SSM and Mamba. The structured state-space sequence model (Fig. 1) is inspired by the particular continuous system, which maps a one-dimensional function or sequence $x(t) \in R^M$ to $y(t) \in R^N$ through the implicit potential state $h(t) \in R^N$, where M represents the number of input sequence channels and N represents the number of state dimensions of SSM. The process of the particular continuous system can be represented as follows:

$$h(t)' = Ah(t) + Bx * (t), y(t) = Ch(t) \quad (1)$$

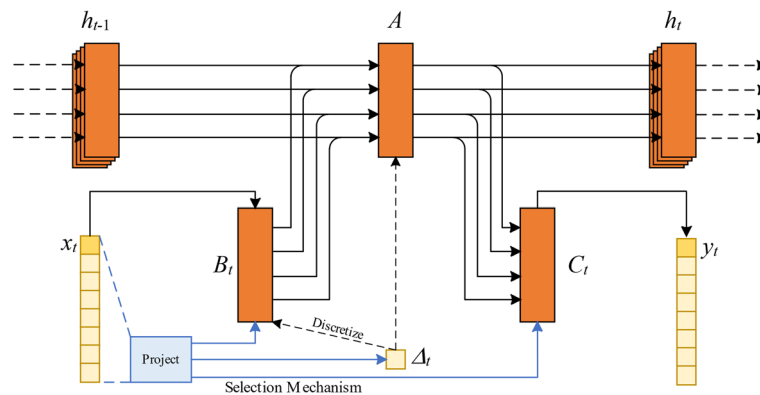


Fig. 1 The overall framework of SSM. SSM maps each channel of input x independently to output y through a higher dimensional latent state h

where $A \in \mathbb{R}^{N \times N}$ and $B, C \in \mathbb{R}^{N \times M}$ are learnable matrices. The continuous sequence is then discretized by a step size Δ . This discretization process can use various discretization rules, such as the zero-order hold shown in Eq. (2):

$$\bar{A} = \exp(\Delta A), \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B \quad (2)$$

where I represents the identity matrix. The discretized SSM model can be expressed as:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, y_t = Ch_t \quad (3)$$

The model is efficiently transformed from the continuous form (Δ, A, B, C) to the discrete form $(\bar{A}, \bar{B}, C)$ by a linear recursive approach. The structured state space model S4 [18] is derived from the classical continuous state space model (SSM). It is implemented using a linear recursive method and initialized with the HiPPO [53] method. The HiPPO [53] method introduces an additional structured component to the state matrix A , effectively improving the modeling ability of long-range dependencies.

Based on S4 [18], Mamba [19] introduces a data-dependent selection mechanism and employs a hardware-aware parallel algorithm to optimize its cyclic processing model further. This mechanism enables Mamba to capture contextual information in long sequences effectively while maintaining high computational efficiency. Compared to the traditional Transformer model, Mamba shows significant performance improvement and higher efficiency in processing long sequence tasks. In the specific operation, Mamba extends the hidden dimension by passing the input sequence through a linear layer and then transforms it into a new representation x' by processing it through the convolution and SiLU activation functions. Next, the state representation y is generated by combining x' with a discrete SSM chosen based on the input parameters. Finally, the final output y is generated by combining y with the residual connections from z that have been processed by the activation function and transformed by linear transformations at time step t . The Mamba structure is shown in Fig. 2. Algorithm 1 describes the operation process of Mamba. In Algorithm 1, the input X is processed through two different linear layers to form two branches. After passing through a Conv1d layer in the first branch, the result is activated using the SiLU function and fed into SSM. The structured state matrix A is obtained within the SSM based on the parameters. The input of the SSM is then passed through two different linear layers to obtain the matrices B and C . The discretization step size Δ is computed, matrices A and B are discretized, and the output of the SSM is calculated. In the second branch, after activating the result using the SiLU function, the activated result multiplies with the output from the first branch and then passes through a linear layer to produce the final output of the algorithm.

Mamba-based channel mixing module

The structure of the CMMamba module is shown in Fig. 2. For the input sequences, we process them separately as follows:

Calculating the similarity scores (left part in the CMMamba module): For each channel sequence $x_i^{1 \times L}$ in the input sequence $X = [x_1, x_2, \dots, x_M]$, compute the cosine similarity scores between the current sequence and the other sequences:

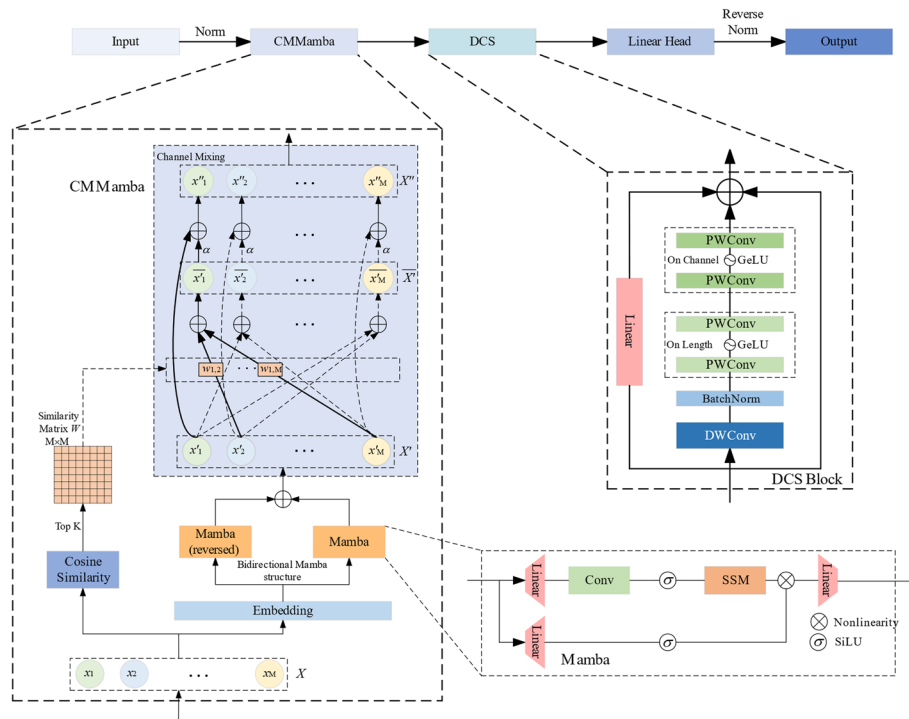


Fig. 2 The overall framework of CMMamba. For ease of understanding, in the channel mixing part, we use solid lines to represent the channel mixing process when the first channel is the target channel and dashed lines to represent the mixing process of the remaining channels

$w_{i,j} = \cos(x_i, x_j)$, where $x_i, x_j \in X$; $i, j = 1, 2, \dots, M$; $i \neq j$. We keep the top K similarity scores with higher similarity and set the rest to 0. And repeating the process for the rest of the sequences to obtain the similarity matrix $W^{M \times M}$. L and M represent sequence length and channel number, respectively.

Algorithm 1 The process of Mamba

-
- | | |
|---|---|
| 1: Input : $X: (B, M, D)$ | B: Batch Size, D: the hidden layer dimension |
| 2: Output : $Y: (B, M, D)$ | E: Linear transformation dimension |
| 3: $x, z: (B, M, E \times D) \leftarrow \text{Linear}(X), \text{Linear}(X)$ | |
| 4: $x': (B, E \times D, M) \leftarrow \text{Transpose}(\text{SiLU}(\text{Conv1D}(x)))$ | |
| 5: $A: (M, N) \leftarrow \text{Parameter}$ | Structured state matrix |
| 6: $B, C: (B, D, N) \leftarrow \text{Linear}(x'), \text{Linear}(x')$ | |
| 7: $\Delta: (B, D, M) \leftarrow \text{Softplus}(\text{Parameter} + \text{Broadcast}(\text{Linear}(x')))$ | |
| 8: $\bar{A}, \bar{B}: (B, D, M, N) \leftarrow \text{discretize}(\Delta, A, B)$ | Input dependent parameters and discretization |
| 9: $y: (B, M, E \times D) \leftarrow \text{Transpose}(\text{SelectiveSSM}(\bar{A}, \bar{B}, C)(x'))$ | |
| 10: $y': (B, M, E \times D) \leftarrow y \otimes \text{SiLU}(z)$ | |
| 11: $Y: (B, M, D) \leftarrow \text{Linear}(y')$ | |
-

Modeling sequences by bidirectional Mamba (lower right part in the CMMamba module): The input sequences are passed through the embedding layer into the bidirectional Mamba structure to obtain a new sequence representation $X' = [x'_1, x'_2, \dots, x'_M]$. The sequences are inverted and passed through the Mamba to achieve reversed Mamba modeling. The subsequence x'_i in X' can be represented as:

$$x'_i = \text{Mamba}(\text{Embedding}(x_i)) + \text{Mamba}_{\text{reversed}}(\text{Embedding}(x_i)), \quad i = 1, 2, \dots, M \quad (4)$$

For each sequence x'_i in the new sequence representation X' , the relevance representation $\overline{x'_i}$ is obtained by multiplying its relevance scores with the corresponding remaining sequences and then summing the results. This process can be formally expressed as:

$$\begin{aligned} \overline{x'_i} &= \sum_{j=1}^M w_{j,i} * x'_j - w_{i,i} * x'_i = w_{1,i} * x'_1 + w_{2,i} * x'_2 + \dots + w_{i-1,i} * x'_{i-1} \\ &\quad + w_{i+1,i} * x'_{i+1} + \dots + w_{M,i} * x'_M, \quad i = 1, 2, \dots, M. \end{aligned} \quad (5)$$

the final sequence x''_i after mixing channels can be formulated as:

$$x''_i = x'_i + \alpha * \overline{x'_i} \quad (6)$$

where the hyperparameter α is the channel mixing weight, which limits the degree of feature mixing. All sequences are denoted as $X'' = [x''_1, x''_2, \dots, x''_M]$. Algorithm 2 describes the computational process of CMMamba block with pseudocode. In Algorithm 2, computing the cosine similarity between each channel and all other channels of the input X is to obtain the similarity matrix W . The Top K similarities in each channel are then retained from W . After embedding the input X , the algorithm performs the following operations for the embedding input: passing through mamba to obtain X_{Mamba} , reversing embedding input along the time dimension and passing through mamba to obtain $X_{\text{Mamba_reversed}}$. The results of these two steps are merged to form X' . The final output of the algorithm is produced by multiplying X' with W , followed by a weighted sum with the matrix X' . The introduction of channel mixing aims to enhance the representation of correlation information between sequences and thus enhance prediction.

Algorithm 2 The process of CMMamba module.

1: Input : $X: (B, L, M)$	L: Senquence Length
2: Output : $X'': (B, D, M)$	
3: $W: (B, M, M) \leftarrow$ Initialize similarity matrix.	
4: $W \leftarrow \text{CosineSimilarity}(x_i, x_j).$	$x_i, x_j \in X. i, j = 1, 2, \dots, M. i \neq j.$
5: $W: (B, M, M) \leftarrow \text{TopK}(W)$	
6: $X_{\text{embedded}}: (B, D, M) \leftarrow \text{EmbeddingLayer}(X)$	
7: $X_{\text{Mamba}}: (B, M, D) \leftarrow \text{Mamba}(\text{Transpose}(X_{\text{embedded}}))$	
8: $X_{\text{Mamba_reversed}}: (B, M, D) \leftarrow \text{Mamba}(\text{Invert}(\text{Transpose}(X_{\text{embedded}})))$	
9: $X': (B, D, M) \leftarrow \text{Transpose}(X_{\text{Mamba}} + X_{\text{Mamba_reversed}})$	
10: $\bar{X}': (B, D, M) \leftarrow X' \cdot W$	
11: $X'': (B, D, M) \leftarrow \alpha \cdot \bar{X}' + X'$	α is a hyperparameter

Deep convolutional structure

Most convolutional models in time series forecasting are devoted to increasing the sensory field without considering convolution in the channel dimension to capture the inter-channel relationships. Inspired by depthwise separable convolution [26], we add an extra set of inter-channel convolutions to deal with inter-channel relationships while using depthwise convolution to learn the temporal dependencies within each channel. The deep convolutional structure is shown in the DCS part of the Fig. 2. Specifically, time dependencies are handled using depthwise separable convolution. Depthwise separable convolution first filters each input channel and then further learns in-sequence features by pointwise convolution. This effectively captures local temporal patterns in the time series while reducing computational complexity and the number of parameters. We then use an additional set of pointwise convolutions in the channel dimension to integrate the results of each channel and learn the channel dependencies. Considering that linear models have the ability to preserve temporal order, we also use residual linear networks to preserve temporal order information on larger time scales. Algorithm 3 describes the calculation process of DCS block. In Algorithm 3, the input is passed through a linear layer to obtain X_L . Then, the algorithm performs the following operations: expanding the input dimensions, applying a depthwise convolution along the expanded dimension and batch normalization. Two pointwise convolutions are then sequentially performed along the length and channel dimensions, reducing the dimensions to obtain the convolutional representation X_{conv} of the input. Finally, X , X_L , and X_{conv} are fused to produce the algorithm output.

Algorithm 3 The process of DCS block.

```

1: Input :  $X: (B, D, M)$ 
2: Output:  $Y: (B, D, M)$ 
3:  $X_L: (B, D, M) \leftarrow \text{Linear}(X)$ 
4:  $X_{conv}: (B, D \times M, I) \leftarrow \text{Reshape}(\text{Extend dimension}(X))$ 
5:  $X_{conv} \leftarrow \text{DWconv}(X_{conv})$ 
6:  $X_{conv}: (B, D, M, I) \leftarrow \text{BatchNorm}(\text{Reshape}(X_{conv}))$ 
7:  $X_{conv}: (B, I, D \times M) \leftarrow \text{PWConv}(\text{GeLU}(\text{PWConv}(\text{Reshape}(X_{conv}))))$ 
8:  $X_{conv} \leftarrow \text{DropOut}(X_{conv})$ 
9:  $X_{conv}: (B, I, M \times D) \leftarrow \text{PWConv}(\text{GeLU}(\text{PWConv}(\text{Reshape}(X_{conv}))))$ 
10:  $X_{conv} \leftarrow \text{DropOut}(X_{conv})$ 
11:  $X_{conv}: (B, D, M) \leftarrow \text{Reduce dimension}(\text{Reshape}(X_{conv}))$ 
12:  $Y: (B, D, M) \leftarrow X + X_L + X_{conv}$ 

```

Experiments

In this section, we conduct many experiments to evaluate the forecasting performance of our proposed model.

Baselines and experiment settings

Baselines. We carefully selected 11 recognized prediction models as our benchmarks, which can be classified into the following four categories:

Mamba-based model: S-Mamba(2024) [52].

Transformer-based models: iTransformer(ICLR2024) [41], PatchTST (ICLR2023) [24] and Crossformer(ICLR2023) [23].

MLP-based models: HDMixer(AAAI2024) [30], TimeMixer(ICLR2024) [54] and DLinear(AAAI2023) [27].

Convolution-based models: MSGNet(AAAI2024) [55], TimesNet(ICLR2023) [46], MICN(ICLR2023) [44] and SCINet(NIPS2022) [45].

Evaluation metrics. We use the mean square error (MSE) and mean absolute error (MAE) as evaluation metrics, which are commonly used in time series forecasting tasks [24, 27, 41]. MSE measures the model performance by calculating the average of the squared differences between the predicted and true values:

Table 1 Details of time series forecasting benchmark datasets

Dataset	ETTh1	ETTh2	ETTm1	ETTm2	Weather	Exchange	ILI
Time steps	17,420	17,420	69,680	69,680	52,696	7588	966
Features	7	7	7	7	21	8	7
Frequency	1 h	1 h	15 min	15 min	10 min	1 day	7 days

$$\text{MSE} = \frac{1}{h} \sum_{i=1}^h (y_i - \hat{y}_i)^2 \# \quad (7)$$

MAE evaluates model performance by calculating the mean of the absolute differences between predicted and true values:

$$\text{MAE} = \frac{1}{h} \sum_{i=1}^h |y_i - \hat{y}_i| \# \quad (8)$$

Datasets. For our experiments, we use seven public datasets commonly used in time series forecasting tasks: ETTh1, ETTh2, ETTm1, ETTm2, Weather, Exchange, and Illness. Table 1 provides detailed properties of the datasets. The following is a brief description of each dataset.

ETT (Electricity Transformer Temperature) contains data collected from a power transformer with seven sensors, including load and oil temperature. It contains two sub-datasets labeled 1 and 2, corresponding to two different power transformers from two different counties in China. Each contains two different resolutions (15 min and 1 h), denoted by m and h. The data are collected from two different counties in China. Thus, we have 4 ETT datasets: ETTh1, ETTh2, ETTm1, and ETTm2. ETT datasets are available at the following link, <https://github.com/zhouhaoyi/ETDataset>. [16]

Weather contains 21 meteorological indicators, such as humidity and temperature, for the whole year of 2020 in Germany. This dataset is available at the following link: <https://www.bgc-jena.mpg.de/wetter/>

Exchange contains daily exchange rates for eight different countries from 1990 to 2016. This dataset is available at the following link: <https://github.com/laiguokun/multivariate-time-series-data>

ILI (Influenza-like Illness) contains weekly recorded data on influenza-like illness (ILI) patients in the United States from 2002 to 2021, with seven indicators, including the number of ILI patients in different age groups and the proportion of ILI patients to total patients. The data were provided by the Centers for Disease Control and Prevention. This dataset is available at the following link: <https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html>

Experiment settings. We adopted the multivariate long-term forecasting settings previously used in PatchTST [24] and DLinear [27]. For the ILI dataset, we set the lookback window to 60 and prediction lengths to {24, 36, 48, 60}. For the remaining datasets, we set the lookback window to 336 and prediction lengths to {96, 192, 336, 720}. All the experiments are implemented in PyTorch [56] and conducted on a single NVIDIA 3080Ti 12 GB GPU.

Main results

Table 2 and 3 demonstrate the excellent performance of CMMamba in long-term forecasting. Specifically, CMMamba achieves most of the best performance in multiple prediction settings across the seven datasets mentioned above. Notably, CMMamba approaches or even surpasses the best available Transformer-based model, PatchTST

Table 2 Multivariate long-term prediction results

Method		CMMamba		HDMixer		TimeMixer		PatchTST		MSGNet		iTransformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.358	0.395	<u>0.375</u>	<u>0.400</u>	0.382	0.407	0.378	0.402	0.390	0.411	0.386	0.405
	192	0.382	0.409	0.414	<u>0.420</u>	0.426	0.427	<u>0.413</u>	0.421	0.442	0.442	0.442	0.436
	336	0.394	<u>0.420</u>	<u>0.397</u>	0.419	0.452	0.445	0.426	0.434	0.480	0.468	0.490	0.461
	720	0.396	0.438	<u>0.457</u>	<u>0.464</u>	0.490	0.484	0.445	<u>0.464</u>	0.494	0.488	0.508	0.493
ETTh2	96	0.233	0.310	<u>0.265</u>	<u>0.331</u>	0.296	0.350	0.275	0.337	0.328	0.371	0.301	0.350
	192	0.281	0.345	<u>0.320</u>	<u>0.370</u>	0.368	0.402	0.338	0.379	0.402	0.414	0.380	0.398
	336	<u>0.322</u>	<u>0.380</u>	0.313	0.373	0.383	0.418	0.329	0.383	0.435	0.443	0.423	0.433
	720	0.353	0.415	0.391	0.426	0.439	0.458	<u>0.380</u>	<u>0.422</u>	0.417	0.441	0.432	0.447
ETTm1	96	0.293	<u>0.347</u>	0.296	0.348	0.305	0.352	<u>0.295</u>	0.345	0.319	0.366	0.341	0.376
	192	0.330	<u>0.371</u>	<u>0.333</u>	0.366	0.350	0.378	0.334	<u>0.371</u>	0.376	0.397	0.383	0.395
	336	<u>0.367</u>	<u>0.389</u>	0.365	0.388	0.382	0.399	<u>0.367</u>	0.392	0.417	0.422	0.418	0.418
	720	<u>0.424</u>	<u>0.420</u>	0.426	0.417	0.451	0.438	0.416	0.421	0.481	0.458	0.488	0.457
ETTm2	96	0.163	0.259	0.167	<u>0.256</u>	0.176	0.264	<u>0.165</u>	0.254	0.177	0.262	0.186	0.272
	192	0.218	0.293	<u>0.222</u>	0.293	0.233	0.301	0.223	<u>0.294</u>	0.247	0.307	0.254	0.314
	336	0.275	0.333	0.279	<u>0.330</u>	0.291	0.338	<u>0.276</u>	0.329	0.312	0.346	0.316	0.351
	720	0.359	<u>0.388</u>	0.366	0.384	0.378	0.394	<u>0.364</u>	0.384	0.414	0.403	0.414	0.407
Weather	96	0.149	0.198	0.162	0.214	<u>0.154</u>	0.204	<u>0.154</u>	<u>0.203</u>	0.163	0.212	0.178	0.219
	192	0.196	0.241	0.205	0.252	0.200	0.247	<u>0.197</u>	<u>0.243</u>	0.212	0.254	0.226	0.259
	336	0.249	0.279	0.242	0.285	<u>0.248</u>	0.286	<u>0.248</u>	<u>0.284</u>	0.272	0.299	0.282	0.300
	720	0.304	0.326	0.321	<u>0.335</u>	0.325	0.338	<u>0.318</u>	<u>0.335</u>	0.350	0.348	0.358	0.350
Exchange	96	<u>0.082</u>	0.197	0.089	0.208	0.102	0.225	<u>0.094</u>	0.215	0.102	0.230	0.086	0.206
	192	0.172	0.292	0.191	<u>0.311</u>	0.203	0.325	<u>0.194</u>	0.314	0.195	0.317	0.180	0.303
	336	<u>0.327</u>	0.409	0.354	0.429	0.346	<u>0.425</u>	<u>0.357</u>	0.435	0.359	0.436	0.338	0.422
	720	0.672	0.589	1.070	0.784	1.010	0.747	<u>1.009</u>	0.760	0.940	0.738	0.858	0.699
ILI	24	1.225	0.657	1.991	0.901	1.726	0.813	<u>1.395</u>	<u>0.743</u>	2.704	1.066	2.145	0.938
	36	1.375	0.723	2.164	0.951	1.660	0.826	<u>1.459</u>	<u>0.790</u>	3.063	1.168	2.272	0.980
	48	1.310	0.749	2.165	0.948	1.851	0.874	<u>1.589</u>	<u>0.804</u>	2.960	1.154	2.239	0.981
	60	1.224	0.708	2.144	0.961	1.648	0.841	<u>1.613</u>	<u>0.804</u>	2.763	1.124	2.327	1.025

The smaller the MAE and MSE scores, the better the performance. The best results are in bold, the second best are underlined

[24], in terms of performance. Compared to models based on the transformer architecture, CMMamba exhibits superior performance, operates faster, and uses less memory (Fig. 3). In addition, Table 4 lists both Mean and standard deviation (STD) for CMMamba and other 3 competitive baselines under 3 different random seeds. It is clear that the STDs are considerably small which indicates the robustness against choice of random seeds of our model. The experimental results demonstrate that our design effectively harnesses Mamba's potential in time series forecasting.

Ablation study

In order to investigate the effectiveness of CMMamba and DCS, we conducted an ablation study on the ETT dataset, and the experimental results are shown in Table 5. After removing the CMMamba component, all the MSE and MAE values increase.

Table 3 This table is a continuation of Table 2

Method		S-Mamba		Dlinear		TimesNet		Crossformer		MICN		SCINet	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.398	0.417	0.378	0.402	0.384	0.402	0.386	0.429	0.396	0.427	0.375	0.406
	192	0.438	0.444	0.415	0.425	0.436	0.429	0.419	0.444	0.430	0.453	0.416	0.421
	336	0.453	0.455	0.447	0.448	0.491	0.469	0.440	0.461	0.433	0.458	0.504	0.495
	720	0.510	0.508	0.481	0.490	0.521	0.500	0.519	0.524	0.474	0.508	0.544	0.527
ETTh2	96	0.301	0.358	0.290	0.350	0.340	0.374	0.628	0.563	0.289	0.357	0.295	0.361
	192	0.366	0.398	0.375	0.410	0.402	0.414	0.703	0.624	0.409	0.438	0.349	0.383
	336	0.391	0.419	0.459	0.462	0.452	0.452	0.827	0.675	0.417	0.452	0.365	0.409
	720	0.417	0.444	0.733	0.606	0.462	0.468	1.181	0.840	0.426	0.473	0.475	0.488
ETTm1	96	0.303	0.358	0.307	0.350	0.338	0.375	0.316	0.373	0.314	0.360	0.325	0.372
	192	0.345	0.383	0.347	0.381	0.374	0.387	0.377	0.411	0.359	0.387	0.354	0.386
	336	0.378	0.403	0.380	0.394	0.410	0.411	0.431	0.442	0.398	0.413	0.394	0.415
	720	0.440	0.440	0.425	0.421	0.478	0.450	0.600	0.547	0.459	0.464	0.476	0.469
ETTm2	96	0.178	0.269	0.167	0.260	0.187	0.267	0.421	0.461	0.178	0.273	0.186	0.281
	192	0.233	0.308	0.223	0.304	0.249	0.309	0.503	0.519	0.245	0.316	0.277	0.356
	336	0.282	0.338	0.291	0.355	0.321	0.351	0.611	0.580	0.295	0.350	0.311	0.369
	720	0.371	0.392	0.407	0.433	0.408	0.403	0.996	0.750	0.389	0.406	0.403	0.412
Weather	96	0.159	0.211	0.168	0.226	0.172	0.220	0.153	0.217	0.161	0.226	0.178	0.233
	192	0.201	0.250	0.211	0.265	0.219	0.261	0.197	0.269	0.220	0.283	0.235	0.277
	336	0.253	0.287	0.257	0.308	0.280	0.306	0.252	0.311	0.275	0.328	0.337	0.345
	720	0.326	0.338	0.315	0.354	0.365	0.359	0.318	0.363	0.311	0.356	0.396	0.413
Exchange	96	0.103	0.229	0.080	<u>0.203</u>	0.107	0.234	0.186	0.346	0.102	0.235	0.116	0.254
	192	0.215	0.339	<u>0.178</u>	0.316	0.226	0.344	0.467	0.522	0.172	0.316	0.218	0.345
	336	0.383	0.457	0.320	0.430	0.367	0.448	0.783	0.721	0.272	0.407	0.294	0.413
	720	1.002	0.762	<u>0.830</u>	<u>0.687</u>	0.964	0.746	1.367	0.943	0.714	0.658	1.110	0.767
ILI	24	2.507	1.043	2.215	1.081	2.317	0.934	3.040	1.186	2.684	1.112	2.150	1.005
	36	2.497	1.061	1.963	0.963	1.972	0.920	3.356	1.230	2.507	1.013	2.103	0.983
	48	2.524	1.070	2.130	1.024	2.238	0.940	3.441	1.223	2.423	1.012	2.432	1.061
	60	2.550	1.094	2.368	1.096	2.027	0.928	3.608	1.302	2.653	1.085	2.325	1.035

This indicates that the CMMamba component is crucial to the model performance, and CMMamba is able to rationally utilize the inter-channel information to help predictions, especially since it is more significant in predicting long-series data. The further increase in MSE and MAE values after removing the DCS component suggests that the DCS component also plays an important role in the model, especially in capturing long-term dependencies. In addition, when DCS is removed, the performance of the model on ETTh1 declines significantly more than on ETTm1, which indicates that the size of the dataset plays a crucial role in Mamba training and performance. A large dataset like ETTm1 allows for more comprehensive learning and generalization. In contrast, a small dataset like ETTh1 may limit the learning potential of Mamba and require additional components or measures to aid in Mamba training.

The study of hyperparameters

Impact of mixing channel quantity on forecast accuracy. We investigated the effect of the number of mixed channels on the prediction. For the ETTh1 dataset, $K \in \{1, 2, \dots, 7\}$, which indicates that only the top K most relevant channels are mixed,

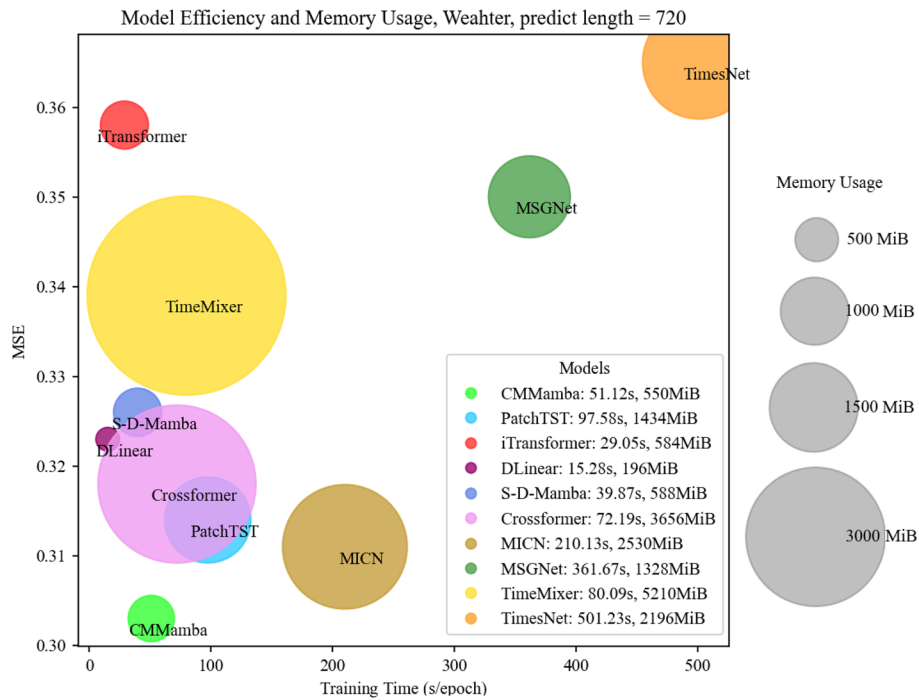


Fig. 3 Comparative experimental results of model efficiency on the Weather dataset and the prediction length is 720

with channel mixing weight $\alpha = 0.2$, and the experimental results are shown in Fig. 4a. For the Weather dataset, $K \in \{1, 3, \dots, 21\}$, $\alpha = 0.3$, and the experimental results are shown in Fig. 4b. From the experimental results, it can be seen that on the ETTh1 dataset, when the value of K is small, feature mixing helps the model capture more helpful information by considering more sequences with high correlation, improving the prediction performance. However, as the K value continues to increase, including too many sequences introduces more irrelevant information. It affects the representation of the features in the target sequences, leading to a decrease in performance. For the ETT datasets, setting $K=2$ is more appropriate.

On the Weather dataset, when $K=1$, the model only considers the one sequence with the highest correlation to the target sequence, which ensures that the features added are the most informative. Appropriate channel features help the model to focus on learning the most essential features and achieve the best prediction. As the value of K increases, more sequences are taken into consideration by the model. Although highly correlated, these sequences may not have all features directly related to the target sequence, which may result in the original information in the target sequence being corrupted or interfered with by non-target information, leading to an increase in error. As the K value increases further, the introduced features may contain more irrelevant or noisy information, disrupting the model's learning of the original vital features. When the model tries to learn from these more extensive ranges of data, it may occasionally be able to capture valuable patterns, leading to performance fluctuations.

Table 4 A subset of the main results including both Mean and STD with 3 runs

Method	CMMamba			HDMixer			TimeMixer			PatchTST		
Metric	MSE	MAE		MSE	MAE		MSE	MAE		MSE	MAE	
ETTh1	96	0.358 ±0.0012	0.395 ±0.0024	<u>0.375</u> ±0.0007	<u>0.400</u> ±0.0002		0.382 ±0.0057	0.407 ±0.0053		0.378 ±0.0033	0.402 ±0.0031	
	192	0.382 ±0.0010	0.409 ±0.0020	0.414 ±0.0009	<u>0.420</u> ±0.0005		0.426 ±0.0021	0.427 ±0.0045		<u>0.413</u> ±0.0007	0.421 ±0.0002	
	336	0.394 ±0.0022	<u>0.420</u> ±0.0015	<u>0.397</u> ±0.0034	0.419 ±0.0013		0.452 ±0.0240	0.445 ±0.0105		0.426 ±0.0006	0.434 ±0.0039	
	720	0.396 ±0.0016	0.438 ±0.0025	<u>0.457</u> ±0.0038	<u>0.464</u> ±0.0007		0.490 ±0.0137	0.484 ±0.0057		0.445 ±0.0029	<u>0.464</u> ±0.0042	
ETTh2	96	0.233 ±0.0004	0.310 ±0.0036	<u>0.265</u> ±0.0013	<u>0.331</u> ±0.0017		0.296 ±0.0055	0.350 ±0.0068		0.275 ±0.0006	0.337 ±0.0001	
	192	0.281 ±0.0012	0.345 ±0.0011	<u>0.320</u> ±0.0013	<u>0.370</u> ±0.0025		0.368 ±0.0108	0.402 ±0.0117		0.338 ±0.0006	0.379 ±0.0004	
	336	<u>0.327</u> ±0.0013	<u>0.380</u> ±0.0022	0.313 ±0.0030	0.373 ±0.0038		0.383 ±0.0071	0.418 ±0.0032		0.329 ±0.0013	0.383 ±0.0022	
	720	0.353 ±0.0022	0.415 ±0.0007	0.391 ±0.0062	0.426 ±0.0066		0.439 ±0.0202	0.458 ±0.0178		<u>0.380</u> ±0.0005	<u>0.422</u> ±0.0003	
ETTrn1	96	0.293 ±0.0016	<u>0.347</u> ±0.0021	0.296 ±0.0008	0.348 ±0.0013		0.305 ±0.0112	0.352 ±0.0041		<u>0.295</u> ±0.0008	0.345 ±0.0024	
	192	0.330 ±0.0021	<u>0.371</u> ±0.0006	<u>0.333</u> ±0.0006	0.366 ±0.0007		0.350 ±0.0153	0.378 ±0.0082		0.334 ±0.0005	0.371 ±0.0013	
	336	<u>0.367</u> ±0.0007	<u>0.389</u> ±0.0017	0.365 ±0.0003	0.388 ±0.0009		0.382 ±0.0068	0.399 ±0.0046		<u>0.367</u> ±0.0021	0.392 ±0.0015	
	720	<u>0.424</u> ±0.0020	<u>0.420</u> ±0.0035	0.426 ±0.0003	0.417 ±0.0009		0.451 ±0.0134	0.438 ±0.0068		0.416 ±0.0020	0.421 ±0.0023	
ETTrn2	96	0.163 ±0.0008	0.259 ±0.0052	0.167 ±0.0005	<u>0.256</u> ±0.0009		0.176 ±0.0016	0.264 ±0.0044		<u>0.165</u> ±0.0009	0.254 ±0.0013	
	192	0.218 ±0.0013	0.293 ±0.0015	<u>0.222</u> ±0.0002	0.293 ±0.0004		0.233 ±0.0053	0.301 ±0.0033		0.223 ±0.0021	<u>0.294</u> ±0.0018	
	336	0.275 ±0.0020	0.333 ±0.0025	0.279 ±0.0015	<u>0.330</u> ±0.0014		0.291 ±0.0060	0.338 ±0.0020		<u>0.276</u> ±0.0013	0.329 ±0.0004	
	720	0.359 ±0.0016	<u>0.388</u> ±0.0038	0.366 ±0.0015	0.384 ±0.0019		0.378 ±0.0091	0.394 ±0.0028		<u>0.364</u> ±0.0016	0.384 ±0.0012	
Weather	96	0.149 ±0.0006	0.198 ±0.0009	0.162 ±0.0025	0.214 ±0.0021		<u>0.154</u> ±0.0061	0.204 ±0.0033		<u>0.154</u> ±0.0038	<u>0.203</u> ±0.0039	
	192	0.196 ±0.0009	0.241 ±0.0008	0.205 ±0.0003	0.252 ±0.0005		0.200 ±0.0061	0.247 ±0.0019		<u>0.197</u> ±0.0024	0.243 ±0.0017	
	336	0.249 ±0.0013	0.279 ±0.0017	0.242 ±0.0006	0.285 ±0.0002		<u>0.248</u> ±0.0030	0.286 ±0.0008		<u>0.248</u> ±0.0023	0.284 ±0.0012	
	720	0.304 ±0.0022	0.326 ±0.0018	0.321 ±0.0007	<u>0.335</u> ±0.0003		0.325 ±0.0096	0.338 ±0.0028		<u>0.318</u> ±0.0029	<u>0.335</u> ±0.0008	
Exchange	96	0.082 ±0.0009	0.197 ±0.0003	<u>0.089</u> ±0.0007	<u>0.208</u> ±0.0011		0.102 ±0.0195	0.225 ±0.0227		0.094 ±0.0008	0.215 ±0.0009	
	192	0.172 ±0.0011	0.292 ±0.0005	<u>0.191</u> ±0.0051	<u>0.311</u> ±0.0051		0.203 ±0.0207	0.325 ±0.0201		0.194 ±0.0031	0.314 ±0.0034	
	336	0.327 ±0.0023	0.409 ±0.0009	0.354 ±0.0080	0.429 ±0.0058		<u>0.346</u> ±0.0081	<u>0.425</u> ±0.0060		0.357 ±0.0121	0.435 ±0.0068	
	720	0.672 ±0.0011	0.589 ±0.0007	1.070 ±0.0203	0.784 ±0.0065		1.010 ±0.0488	0.747 ±0.0130		<u>1.009</u> ±0.0691	0.760 ±0.0315	
ILI	24	1.225 ±0.0293	0.657 ±0.0169	1.991 ±0.0286	0.901 ±0.0104		1.726 ±0.0402	0.813 ±0.0098		<u>1.395</u> ±0.0541	<u>0.743</u> ±0.0080	
	36	1.375 ±0.0478	0.723 ±0.0141	2.164 ±0.0583	0.951 ±0.0128		1.660 ±0.0277	0.826 ±0.0081		<u>1.459</u> ±0.0227	<u>0.790</u> ±0.0329	
	48	1.310 ±0.0559	0.749 ±0.0195	2.165 ±0.0210	0.948 ±0.0057		1.851 ±0.0241	0.874 ±0.0091		<u>1.589</u> ±0.0784	0.804 ±0.0151	
	60	1.224 ±0.0782	0.708 ±0.0225	2.144 ±0.0186	0.961 ±0.0068		1.648 ±0.0221	0.841 ±0.0084		<u>1.613</u> ±0.1983	<u>0.804</u> ±0.0526	

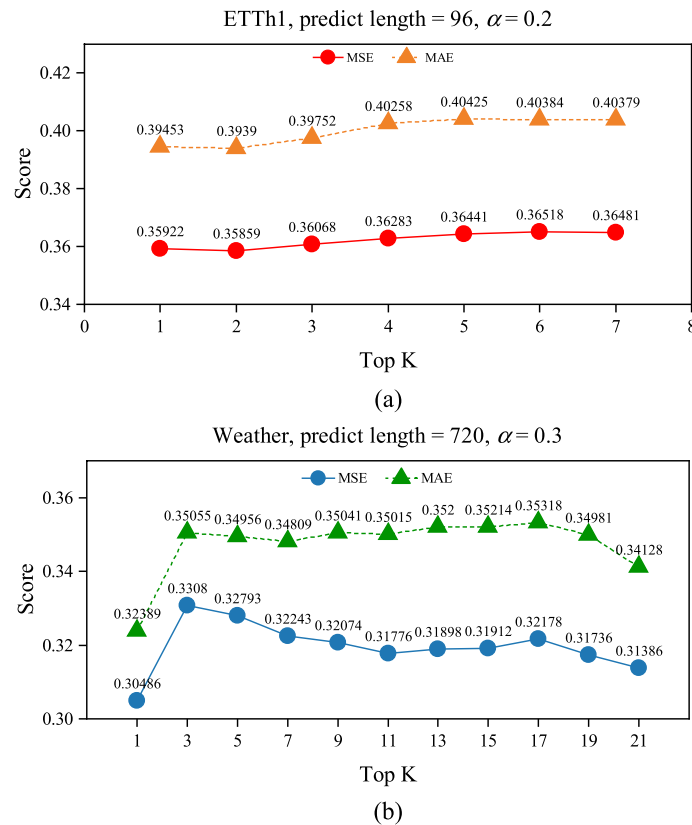
The best results are in bold, the second best are underlined

Table 5 Results of ablation experiments on the ETTh1 and ETTm1 datasets

	Dataset	Predict Length	ETTh1				ETTh1			
			96	192	336	720	96	192	336	720
wo CMMamba	MSE		0.406	0.420	0.431	0.500	0.341	0.379	0.390	0.441
	MAE		0.435	0.445	0.459	0.514	0.382	0.403	0.406	0.437
wo DCS	MSE		0.691	0.704	0.686	0.723	0.323	0.363	0.396	0.484
	MAE		0.554	0.577	0.574	0.608	0.371	0.394	0.409	0.478

'wo CMMamba' and 'wo DCS' indicates training without CMMamba and DCS modules respectively

Impact of the channel mixing weight on forecast accuracy. We studied the effect of channel mixing weight on the predictions. In order to make the results more discriminative, we let K is set to half of the number of channels in the dataset. For the ETTh1 dataset, we set $K=4$, and the channel mixing weights $\alpha \in \{0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5\}$, and the results of the experiments are shown in Fig. 5a. For the Weather dataset, we set $K=11$ and $\alpha \in \{0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5\}$, and the experimental results are shown in Fig. 5b. For the ETTh1 dataset, the model performance is the best when α is set to 0.05. It is sub-optimal when $\alpha \in \{0.01, 0.1\}$, but the error does not increase significantly. On the Weather dataset, the model performs best when α is set to 0.01. It is sub-optimal when $\alpha \in \{0.001, 0.01, 0.05, 0.1, 0.2\}$ with a slight decrease in performance. These

**Fig. 4** Impact of mixing channel quantity K on CMMamba forecast accuracy on **a** ETTh1 and **b** Weather datasets

results suggest that moderate information enhancement helps the model capture and utilize the features of the correlated series better, thus improving the forecasting accuracy. When the value of α increases, the model performance decreases significantly on both datasets because the excessive weighting makes the model overemphasize the features of other channels. At the same time, it ignores the equally important information in the target channel. This significantly affects the feature representations of the original channel and thus affects the accuracy of the prediction.

Impact of the input length on forecast accuracy. Finally, we set $K=2$, channel mixing weight $\alpha = 0.2$, and select different input lengths to investigate how the input length affects the channel mixing. The experimental results are shown in Fig. 6. The experimental results show that the effect of channel mixing is significantly affected by the input length. At shorter sequence lengths, the model acquires limited contextual information, which restricts the model to capture long-term dependencies. In this case, the effect of channel mixing is not significant because the model struggles to learn enough inter-channel dynamic changes from the limited input. As the input length increases, the model's ability to capture temporal dependencies increases, which makes feature integration via channel mixing more effective. However, when the input length is too long, the channel mixing features contain too much early information in other channels. This information is irrelevant to the future target values to

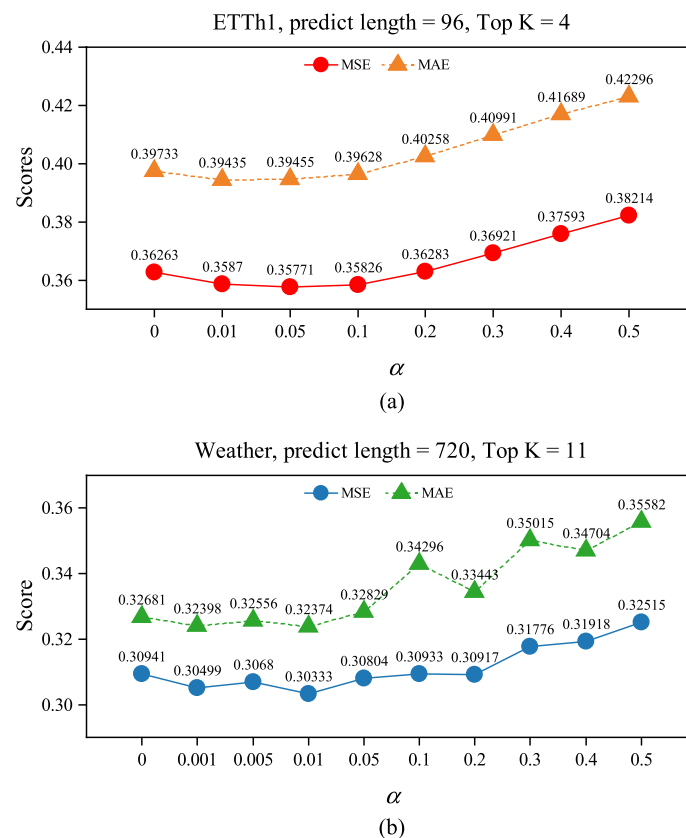


Fig. 5 Impact of the channel mixing weight α on CMMamba forecast accuracy on **a** ETTh1 and **b** Weather datasets

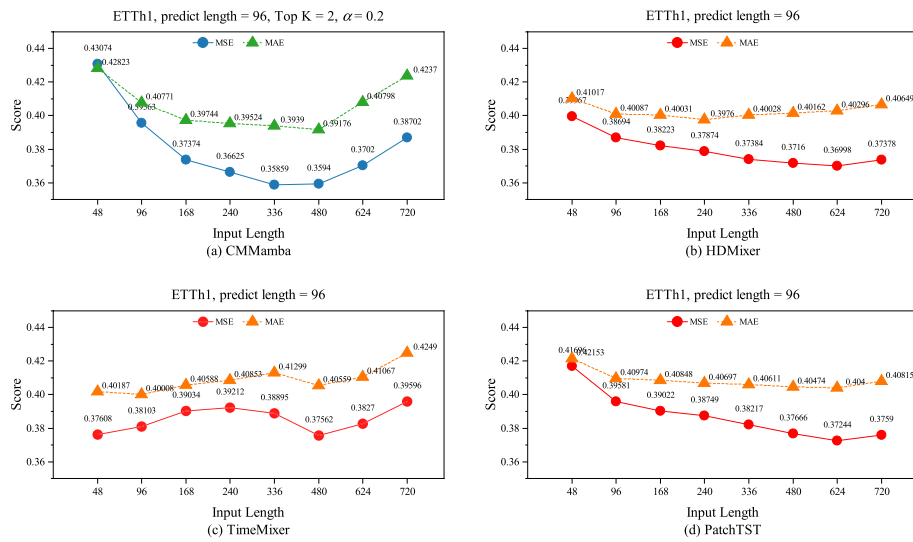


Fig. 6 Impact of the input length on the forecast accuracy of different baselines on the ETTh1 dataset: **a** CMMamba, **b** HDMixer, **c** TimeMixer, **d** PatchTST

be predicted in the target channel and, when added, affects the representation of the features within the target channel and, hence, the prediction performance. For the ETT datasets, 336 is a suitable input length for channel mixing.

Conclusion and prospect

In this study, we propose CMMamba. CMMamba efficiently captures the temporal dependencies within a channel by utilizing a bidirectional Mamba structure. In addition, CMMamba uses a channel mixing mechanism to control the information fusion and avoids the interference of low-correlation features. CMMamba improves the computational efficiency of long sequence prediction tasks while maintaining performance. We also propose a deep convolutional structure to enhance further the model's ability to learn cross-channel dependencies and temporal order information. Extensive experiments on seven publicly available datasets demonstrated the superiority of CMMamba.

Besides the field of time series prediction, we also hope that this work can provide some references and research ideas for other fields. For example, in the field of CV, the method proposed in this study can be used to improve motion [57, 58] and trajectory prediction [59]. For video content analysis, the proposed method can also be used to improve feature integration for action recognition [60] and anomaly detection [61].

In this work, we noticed that Mamba is more effective in modeling long sequences, but its ability to model short sequences still needs to be improved, and this problem is also reflected in other fields [62]. Therefore, the next step could be to consider improving Mamba to enhance its ability to model short sequences.

Acknowledgements

Not applicable.

Author contributions

QL was a major contributor in writing the manuscript. JWQ, DCW, DSC and DZS reviewed the manuscript and suggested changes. All authors read and approved the final manuscript.

Funding

This work was supported by the Key Research and Development Program of Xinjiang Uygur Autonomous Region: Research and Development of Key Technologies for Calculation, Measurement, and Digitized Management of Carbon Emission and Carbon Sink Indicators in the Energy Sector (NO.2022B01010), Outstanding Doctoral Student Innovation Project of Xinjiang University (NO.XJU2024BS085).

Availability of data and materials

No datasets were generated or analysed during the current study.

Declarations**Ethics approval and consent to participate**

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 29 May 2024 Accepted: 22 September 2024

Published online: 29 October 2024

References

1. Lim B, Zohren S. Time-series forecasting with deep learning: a survey. *Phil Trans R Soc A*. 2021;379(2194):20200209.
2. Chai SH, Lim JS, Yoon H, Wang B. A novel methodology for forecasting business cycles using Arima and neural network with weighted fuzzy membership functions. *Axioms*. 2024;13(1):56.
3. Ding Y, He D, Wu J, Xu X. Crude oil spot price forecasting using Ivanov-based lasso vector autoregression. *Complexity*. 2022;2022(1):5011174.
4. Shi Y. Forecasting mortality rates with the penalized exponential smoothing state space model. *J Oper Res Soc*. 2022;73(5):955–68.
5. Han Z, Zhao J, Leung H, Ma KF, Wang W. A review of deep learning models for time series prediction. *IEEE Sens J*. 2019;21(6):7833–48.
6. Wu X, Wang Y. Extended and unscented Kalman filtering based feedforward neural networks for time series prediction. *Appl Math Model*. 2012;36(3):1123–31.
7. Zhang X, Ding C, Wang G. An autoregressive-based Kalman filter approach for daily pm_{2.5} concentration forecasting in Beijing, China. *Big Data*. 2024;12(1):19–29.
8. Bengio Y, LeCun Y. Scaling learning algorithms toward AI. In: Bottou L, Chapelle O, DeCoste D, Weston J, editors. *Large-scale kernel machines*. The MIT Press; 2007. p. 321–60. <https://doi.org/10.7551/mitpress/7496.003.0016>.
9. Urrea C, Agramonte R. Kalman filter: historical overview and review of its use in robotics 60 years after its creation. *J Sens*. 2021;2021(1):9674015.
10. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, et al. Language models are few-shot learners. *Adv Neural Inf Process Syst*. 2020;33:1877–901.
11. Kalyan KS, Rajasekharan A, Sangeetha S. Ammu: a survey of transformer-based biomedical pretrained language models. *J Biomed Inform*. 2022;126: 103982.
12. Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Housley N. An image is worth 16x16 words: transformers for image recognition at scale. In: *International conference on learning representations*. 2021. <https://openreview.net/forum?id=YicbFdNTTy>.
13. Rao Y, Zhao W, Zhu Z, Lu J, Zhou J. Global filter networks for image classification. *Adv Neural Inf Process Syst*. 2021;34:980–93.
14. Khan S, Naseer M, Hayat M, Zamir SW, Khan FS, Shah M. Transformers in vision: a survey. *ACM Comput Surv (CSUR)*. 2022;54(10s):1–41.
15. Kitaev N, Kaiser L, Levskaya A. Reformer: the efficient transformer. *arXiv preprint arXiv:2001.04451*. 2020.
16. Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W. Informer: beyond efficient transformer for long sequence time-series forecasting. In: *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, pp. 11106–11115 (2021).
17. Gu A, Johnson I, Goel K, Saab KK, Dao T, Rudra A, Re C. Combining recurrent, convolutional, and continuous-time models with linear state space layers. In: Beygelzimer A, Dauphin Y, Liang P, Vaughan JW, editors. *Advances in Neural Information Processing Systems*. 2021. <https://openreview.net/forum?id=yWd42CWN3c>.
18. Gu A, Goel K, Re C. Efficiently modeling long sequences with structured state spaces. In: *International conference on learning representations*. 2022. <https://openreview.net/forum?id=uYLFOz1vIAC>.
19. Gu A, Dao T. Mamba: linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*. 2023.
20. Xu Z, Lin Y, Han H, Yang S, Li R, Zhang Y, Li X. Mambatalk: efficient holistic gesture synthesis with selective state space models. *CoRR abs/2403.09471*. 2024.

21. He H, Bai Y, Zhang J, He Q, Chen H, Gan Z, Wang C, Li X, Tian G, Xie L. Mambaad: exploring state space models for multi-class unsupervised anomaly detection. *CoRR* abs/2404.06564. 2024.
22. Archit A, Pape C. Vim-UNet: vision mamba for biomedical segmentation. In: Medical imaging with deep learning. 2024. <https://openreview.net/forum?id=PYNwysgFeP>.
23. Zhang Y, Yan J. Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. In: The eleventh international conference on learning representations. 2023. <https://openreview.net/forum?id=vSvLM2j9eie>.
24. Nie Y, Nguyen NH, Sinthong P, Kalagnanam J. A time series is worth 64 words: long-term forecasting with transformers. In: The eleventh international conference on learning representations. 2023. <https://openreview.net/forum?id=Jbdc0vTOcol>.
25. Han W, Zhu T, Chen L, Ning H, Luo Y, Wan Y. Mcformer: multivariate time series forecasting with mixed-channels transformer. *IEEE Internet Things J*. 2024.
26. Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H. Mobilenets: efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*. 2017.
27. Zeng A, Chen M, Zhang L, Xu Q. Are transformers effective for time series forecasting? In: Proceedings of the AAAI conference on artificial intelligence, vol. 37, pp. 11121–11128. 2023.
28. Box GE, Jenkins GM, Reinsel GC, Ljung GM. Time series analysis: forecasting and control. New Jersey: Wiley; 2015.
29. Ray W. Time series: theory and methods. Wiley Online Library. 1990.
30. Huang Q, Shen L, Zhang R, Cheng J, Ding S, Zhou Z, Wang Y. Hdmixer: hierarchical dependency with extendable patch for multivariate time series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 38, pp. 12608–12616. 2024.
31. Palaskar S, Ekambaram V, Jati A, Gantayat N, Saha A, Nagar S, Nguyen NH, Dayama P, Sindhgatta R, Mohapatra P, et al. Automixer for improved multivariate time-series forecasting on business and it observability data. In: Proceedings of the AAAI conference on artificial intelligence, vol. 38, pp. 22962–22968. 2024.
32. Oreshkin BN, Carpov D, Chapados N, Bengio Y. N-beats: neural basis expansion analysis for interpretable time series forecasting. In: International conference on learning representations. 2020. <https://openreview.net/forum?id=r1ecqn4YwB>.
33. Fan W, Zheng S, Yi X, Cao W, Fu Y, Bian J, Liu T-Y. DEPTS: deep expansion learning for periodic time series forecasting. In: International conference on learning representations. 2022.
34. Zhang T, Zhang Y, Cao W, Bian J, Yi X, Zheng S, Li J. Less is more: fast multivariate time series forecasting with light sampling-oriented MLP structures. *arXiv preprint arXiv:2207.01186*. 2022.
35. Challu C, Olivares KG, Oreshkin BN, Ramirez FG, Canseco MM, Dubrawski A. Nhits: neural hierarchical interpolation for time series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 37, pp. 6989–6997. 2023.
36. Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang Y-X, Yan X. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Adv Neural Inf Process Syst*. 2019;32.
37. Wu H, Xu J, Wang J, Long M. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. *Adv Neural Inf Process Syst*. 2021;34:22419–30.
38. Liu S, Yu H, Liao C, Li J, Lin W, Liu AX, Dustdar S. Pyraformer: low-complexity pyramidal attention for long-range time series modeling and forecasting. In: International conference on learning representations. 2021.
39. Zhou T, Ma Z, Wen Q, Wang X, Sun L, Jin R. Fedformer: frequency enhanced decomposed transformer for long-term series forecasting. In: International conference on machine learning, PMLR. pp. 27268–27286. 2022.
40. Du D, Su B, Wei Z. Preformer: predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting. In: ICASSP 2023—2023 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp. 1–5 (2023). <https://doi.org/10.1109/ICASSP49357.2023.10096881>.
41. Liu Y, Hu T, Zhang H, Wu H, Wang S, Ma L, Long M. itransformer: Inverted transformers are effective for time series forecasting. In: The Twelfth International Conference on Learning Representations. 2024. <https://openreview.net/forum?id=JePfA18fah>.
42. Bai S, Kolter JZ, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*. 2018.
43. Sen R, Yu H-F, Dhillon IS. Think globally, act locally: a deep neural network approach to high-dimensional time series forecasting. *Adv Neural Inf Process Syst*. 2019;32.
44. Wang H, Peng J, Huang F, Wang J, Chen J, Xiao Y. MICN: multi-scale local and global context modeling for long-term series forecasting. In: The eleventh international conference on learning representations. 2023. <https://openreview.net/forum?id=zt53IDUR1U>.
45. Liu M, Zeng A, Chen M, Xu Z, Lai Q, Ma L, Xu Q. SCINet: time series modeling and forecasting with sample convolution and interaction. In: Oh AH, Agarwal A, Belgrave D, Cho K, editors. Advances in neural information processing systems. 2022. <https://openreview.net/forum?id=AyajSJTAzmG>.
46. Wu H, Hu T, Liu Y, Zhou H, Wang J, Long M. Timesnet: temporal 2d-variation modeling for general time series analysis. In: International conference on learning representations. 2023.
47. Pham Q, Liu C, Sahoo D, Hoi S. Learning fast and slow for online time series forecasting. In: The eleventh international conference on learning representations. 2023. <https://openreview.net/forum?id=q-PbPHD3EOk>.
48. Yi K, Zhang Q, Fan W, He H, Hu L, Wang P, An N, Cao L, Niu Z. FourierGNN: rethinking multivariate time series forecasting from a pure graph perspective. In: Thirty-seventh conference on neural information processing systems. 2023. <https://openreview.net/forum?id=bGs1qWQ1Fx>.
49. Shen L, Kwok J. Non-autoregressive conditional diffusion models for time series prediction. In: International conference on machine learning, PMLR; 2023. pp. 31016–31029.
50. Kolloviev M, Ansari AF, Bohlke-Schneider M, Zschiegner J, Wang H, Wang B. Predict, refine, synthesize: self-guiding diffusion models for probabilistic time series forecasting. In: Thirty-seventh conference on neural information processing systems. 2023. <https://openreview.net/forum?id=q6X038vKgU>.

51. Zhou L, Poli M, Xu W, Massaroli S, Ermon S. Deep latent state space models for time-series generation. In: International conference on machine learning. PMLR;2023. pp. 42625–42643.
52. Wang Z, Kong F, Feng S, Wang M, Zhao H, Wang D, Zhang Y. Is mamba effective for time series forecasting? arXiv preprint [arXiv:2403.11144](https://arxiv.org/abs/2403.11144). 2024.
53. Gu A, Dao T, Ermon S, Rudra A, Re C. Hippo: recurrent memory with optimal polynomial projections. *Adv Neural Inf Process Syst*. 2020;33:1474–87.
54. Wang S, Wu H, Shi X, Hu T, Luo H, Ma L, Zhang JY, Zhou J. Timemixer: decomposable multiscale mixing for time series forecasting. In: International conference on learning representations (ICLR). 2024.
55. Cai W, Liang Y, Liu X, Feng J, Wu Y. Msgnet: learning multi-scale inter-series correlations for multivariate time series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 38, pp. 11141–11149, 2024.
56. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L et al. Pytorch: An imperative style, high-performance deep learning library. *Adv Neural Inf Process Syst*. 2019;32.
57. Xu C, Tan RT, Tan Y, Chen S, Wang YG, Wang X, Wang Y. Eqmotion: equivariant multi-agent motion prediction with invariant interaction reasoning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 1410–1420, 2023.
58. Jiang C, Cornman A, Park C, Sapp B, Zhou Y, Anguelov D, et al. Motiondiffuser: controllable multi-agent motion prediction using diffusion. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 9644–9653, 2023.
59. Alahi A, Goel K, Ramanathan V, Robicquet A, Fei-Fei L, Savarese S. Social lstm: human trajectory prediction in crowded spaces. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 961–971, 2016.
60. Su Y, Xing M, An S, Peng W, Feng Z. Vdarn: video disentangling attentive relation network for few-shot and zero-shot action recognition. *Ad Hoc Netw*. 2021;113: 102380.
61. Su Y, Tan Y, An S, Xing M. Anomalies cannot materialize or vanish out of thin air: a hierarchical multiple instance learning with position-scale awareness for video anomaly detection. *Expert Syst Appl*. 2024;254: 124392.
62. Yu W, Wang X. Mambaout: do we really need mamba for vision? arXiv preprint [arXiv:2405.07992](https://arxiv.org/abs/2405.07992). 2024.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.